

Assurance Assignment: Hand-in

Role: SAMM specialist

Francesco Maria Fuligni | Group 13

Core

Security Testing (Verification)

Dedicated security test cases for known-risk areas, run regularly. I audited authorization, path-traversal and login-throttling tests, plus `server/test/api/test_FCS_COP_1.py`. Its existing checks covered an issued link returning HTTP 200/PDF without authentication; random, last-character-flipped, plus-one and all-zero unissued links returning HTTP 404/no PDF; and two users' links retrieving only their own documents and watermark secrets. `.github/workflows/tests.yml` runs pytest on PRs to `main`, pushes to `main` or `feat/**`, and manual dispatch.

Improvement

Before, the SHA-1(s1) candidate was checked only manually, and the four negative-link cases did not check for sensitive values in responses. I automated the exact candidate request below, which returns HTTP 404 without PDF content:

```
GET /api/get-version/640d87e741e6aa4c669a82a4cd304787960513ab
```

I also added assertions that all four negative responses omit both users' watermark secrets, intended recipients and filenames. The focused module passed all seven cases and Ruff on 30 September 2026. Commit: `fbae18a`.

Scorecard

Level 2, using the assignment rubric. Dedicated tests for known risks run regularly in CI; the improvement adds repeatable checks within that level. Level 3 requires systematic coverage tracking, which is missing. The link fixture uses in-memory SQLite and `toy-eof`; these selected cases do not establish general link unpredictability.

Further Work

Secure Build (Implementation) – second practice audited

Builds automated and reproducible; dependencies pinned. I audited the Dockerfile, dependency files, Compose configuration and CI. Before the improvement, CI did not build the image and Docker ignored the local `uv.lock`: the built image had `cryptography 50.0.2`, while the lockfile specified `50.0.1`. A manual image build/import and `docker compose config --quiet` passed, but did not establish a pinned build or startup of a fresh Compose stack.

Improvement – second real improvement

I added a CI job that builds `./server` and runs `import rmap` inside the image. I also added `server/requirements.txt`, generated with `pip-compile`, pinning 26 registry distributions. Docker installs that file and the project once with `--no-deps`; pytest uses the same runtime requirements. CI regenerates the file and fails on a diff; the Python base image is pinned by digest. A clean local Docker build passed, all 26 installed versions matched, and `pip check` and `import rmap` passed. Recorded CI run `36756093546` passed the lock check, Docker job and Ruff; pytest reported 368 passed, 11 skipped, 2 xfailed. Commits: `8a1073f`, `f0cd7ae`, `f7e50c3`.

Scorecard

Level 2, using the assignment rubric, for the server image and Python runtime dependencies. CI automates build/import checks, installs tracked exact versions and checks for dependency drift. System packages remain unpinned, RMAP uses a release tag, and fresh full-stack startup was not tested. Level 3 requires a security gate: no dedicated dependency/image vulnerability scan or confirmed required merge check is established.

Hours

Track 2 time spent (self-reported): 9 hours (5 Core + 4 Further Work).